All-Pairs Shortest Paths with Fox's Algorithm

Min-plus repeated squaring distributed with MPI

Faculty of Sciences · University of Porto

Antonio Lanção (up202205416)
David Sá (up202303580)
Henrique Teixeira (up202306640)

Parallel Computing 2026/27
24 October 2026

Abstract. We solve the all-pairs shortest path problem for directed weighted graphs by repeated squaring of the distance matrix under the min-plus semiring. Each product is distributed over a $Q \times Q$ process grid with Fox's algorithm, implemented in C with MPI. The communication of each step overlaps the computation of the previous one, and the iteration stops as soon as the distances no longer change. [TODO: one sentence with the headline result.]

1 Introduction

Given a directed graph with N nodes and non-negative edge weights, the all-pairs shortest path (APSP) problem asks for the length of the shortest path between every ordered pair of nodes. The graph is given as an $N \times N$ adjacency matrix W, where W_{ij} is the weight of the edge $i \to j$ and 0 off the diagonal means there is no edge.

This report describes a sequential baseline and a parallel implementation using Fox's algorithm over MPI, how both were validated, and how they perform on the LabCC cluster.

2 Algorithm

2.1 Min-plus product

The min-plus product of two $N \times N$ matrices replaces the sum by min and the product by $+$:

$$(A \otimes B)_{ij} = \min_{k=1}^{N} \big(A_{ik} + B_{kj}\big) \tag{1}$$

If $D^{(m)}$ holds the shortest distances using at most m edges, then $D^{(2m)} = D^{(m)} \otimes D^{(m)}$. Starting from $D^{(1)} = W$ (with ∞ for missing edges and 0 on the diagonal), $\lceil \log_2(N-1) \rceil$ squarings are enough, since a shortest path has at most $N-1$ edges. The total cost is $O(N^3 \log N)$, a $\log N$ factor above Floyd–Warshall, but every step is a matrix product, which parallelises well.

Both implementations also stop early: if a squaring leaves the matrix unchanged, every later squaring would too, so the current matrix is already the answer.

2.2 Fox's algorithm

The $P = Q^2$ processes form a $Q \times Q$ grid, and process (i, j) owns block (i, j) of A, B and C, each of size $N/Q \times N/Q$. Fox's algorithm computes $C = A \otimes B$ in Q steps:

```
for s = 0 .. Q-1:
    k = (i + s) mod Q
    broadcast A[i][k] along row i          # process (i, k) is the root
    C[i][j] = min(C[i][j], A[i][k] (x) B[k][j])
    shift B one row up (circularly) along column j
```

After step s, process (i, j) holds $B_{(i+s+1) \bmod Q, j}$, which is exactly the block it needs for the next value of k. Each process performs $Q \cdot (N/Q)^3 = N^3/P$ operations per product and, per step, receives one A block through a row broadcast and one B block from its neighbour.

3 Implementation

The code is split into a library without MPI (`lib/common`, `lib/seq`) and the distributed part (`lib/mpi`). A single `src/main.c` is compiled twice, as `seq` and, with `-DUSE_MPI`, as `fox`.

3.1 Data representation

Distances are stored as `uint32_t` in a row-major array. Missing edges are stored as $\mathrm{INF} = (2^{32} - 1)/2$, so the sum of two values that are each at most INF cannot overflow. On output, INF is printed as 0, as the assignment requires. Reading the input forces the diagonal to 0 and maps off-diagonal zeros to INF.

3.2 Sequential kernel

The min-plus kernel (`minplus_relax`) accumulates $C_{ij} \leftarrow \min\bigl(C_{ij}, A_{ik} + B_{kj}\bigr)$, so calling it once per Fox step builds the full product. Three choices make it fast:

- **Loop order *i–k–j*.** The inner loop walks along a row of B and a row of C, both contiguous, so the compiler vectorises it with SIMD `min` instructions. With `-O3 -march=native` this is 8 lanes on AVX2.
- **Tiling.** The three loops are blocked in 128×128 tiles, so the parts of A, B and C in use stay in cache.
- **Skipping unreachable rows.** If $A_{ik} = \mathrm{INF}$, row k of B cannot improve row i of C and is skipped. Sparse inputs benefit most.

The `restrict` qualifiers tell the compiler that the three matrices do not alias. `floyd_warshall` is kept only as an independent reference for the unit tests.

3.3 Process grid and distribution

Rank 0 reads the matrix and broadcasts N. Every rank then checks that P is a perfect square and that Q divides N. Otherwise rank 0 prints `ERROR: Invalid configuration!` and all ranks exit. The grid is a periodic two-dimensional Cartesian communicator (`MPI_Cart_create`, created without reordering so that grid rank 0 is the rank that read the input), and `MPI_Cart_sub` derives one communicator per row and one per column.

Blocks are not contiguous in the global matrix, so rank 0 packs them in rank order into a buffer and distributes them with a single `MPI_Scatter`. The result is collected with `MPI_Gather` and unpacked in place into the input matrix.

3.4 Overlapping communication and computation

The transfers for step $s + 1$ start before the computation of step s (`fox_multiply`):

- the next A block is broadcast with a non-blocking `MPI_Ibcast` into one of two alternating buffers, while the current one is in use;
- the B shift is an `MPI_Isend` of the current block and an `MPI_Irecv` into a spare buffer, and the two buffers are swapped after `MPI_Waitall`.

The computation only reads the buffers being sent, which MPI allows, so the step costs roughly $\max\bigl(t_{\mathrm{comp}}, t_{\mathrm{comm}}\bigr)$ instead of their sum. The process that is the broadcast root sends directly from its own A block, without a copy. All buffers are allocated once, in `mpi_apsp`, and reused across squarings. Each process holds six blocks, i.e. $6(N/Q)^2 \cdot 4$ bytes.

3.5 Early termination

After each product, every process compares its new block with the previous one, and an `MPI_Allreduce` with `MPI_LOR` decides whether any block changed. The cost is one small collective per squaring, which can save several full products. [TODO: report how many squarings each input actually needs.]

3.6 Shared-copy variant

`fox_shared` is the same algorithm with the communication inside the product removed. Every process keeps a full copy of the matrix, stored block by block in rank order, so block (i, j) is the $(iQ + j)$ -th contiguous $N/Q \times N/Q$ piece. Because every process knows where each block is, step s of the product reads A_{ik} and B_{kj}, with $k = (i + s) \bmod Q$, straight from memory (`fox_block` returns a view, not a copy), instead of receiving them by row broadcast and column shift:

```
for s = 0 .. Q-1:
    k = (i + s) mod Q
    C[i][j] = min(C[i][j], M[i][k] (x) M[k][j])     # M: local copy
```

Each squaring still needs the whole new matrix everywhere, so after the product a single `MPI_Allgather` writes every process's new block into its place in every copy. The block layout makes this possible without packing. The matrix is broadcast once at the start and unpacked on rank 0 at the end. Only `mpi_apsp` differs between the two binaries; `fox_shared` is the same source compiled with `-DFOX_SHARED`.

The trade-off is volume and memory. Per squaring, `fox` delivers $2Q$ blocks to each process, $2N^2/Q$ values, in $2Q$ messages that overlap the computation. `fox_shared` delivers N^2 values to each process in one collective that does not overlap anything, $Q/2$ times more data, and each process holds $N^2 + (N/Q)^2$ values instead of $6(N/Q)^2$.

4 Validation

- **Unit tests** (`make test`): matrix allocation and copy, input parsing (including malformed input), and the min-plus kernel. The kernel is checked against the assignment's example, the identity matrix, all-INF matrices (overflow), split accumulation, and Floyd–Warshall on a random graph with $N = 150$, which is not a multiple of the tile size.
- **Regression** (`make check`): `seq` and `fox` are compared byte for byte with the reference outputs for $N \in \{6, 300, 600, 900, 1200, 3000\}$, with $P \in \{1, 4, 9, 16\}$. Configurations where Q does not divide N must be rejected with exactly the expected error message.

All tests pass.

5 Performance evaluation

5.1 Methodology

[TODO: LabCC node specification: CPU, cores per node, memory, network, compiler and MPI versions, and how processes were mapped to nodes.]

Times are wall-clock times of the computation only (scatter, squarings and gather), measured on rank 0 with `MPI_Wtime`. The clock starts after an `MPI_Barrier` and stops once the gather returns. Input and output are excluded. Each configuration was run [TODO: R] times and the minimum is reported. The benchmark is driven by `bench/run_bench.sh`. Speedup and efficiency are computed relative to the sequential program:

$$S(P) = \frac{T_{\text{seq}}}{T(P)}, \quad E(P) = \frac{S(P)}{P} \tag{2}$$

5.2 Results

Table 1. Execution time in milliseconds.

N	seq	$P = 1$	$P = 4$	$P = 9$	$P = 16$	$P = 25$
300						
600						
900						
1200						
3000						

Table 2. Speedup relative to the sequential version.

N	$P = 1$	$P = 4$	$P = 9$	$P = 16$	$P = 25$
300					
600					
900					
1200					
3000					

5.3 Fox against the shared-copy variant

Figure 1 and Table 3 compare `fox` and `fox_shared` on random matrices with $N = 2^6, \ldots, 2^{10}$ (`matgen`, seed 42), measured with `bench/run_power2.sh` on a laptop with an AMD Ryzen 7 7840HS (8 cores, 16 hardware threads). Values are the median of 3 runs (6 for $P \in \{4, 16\}$). $P = 64$ oversubscribes the machine, four processes per hardware thread, so those points show contention, not scaling.

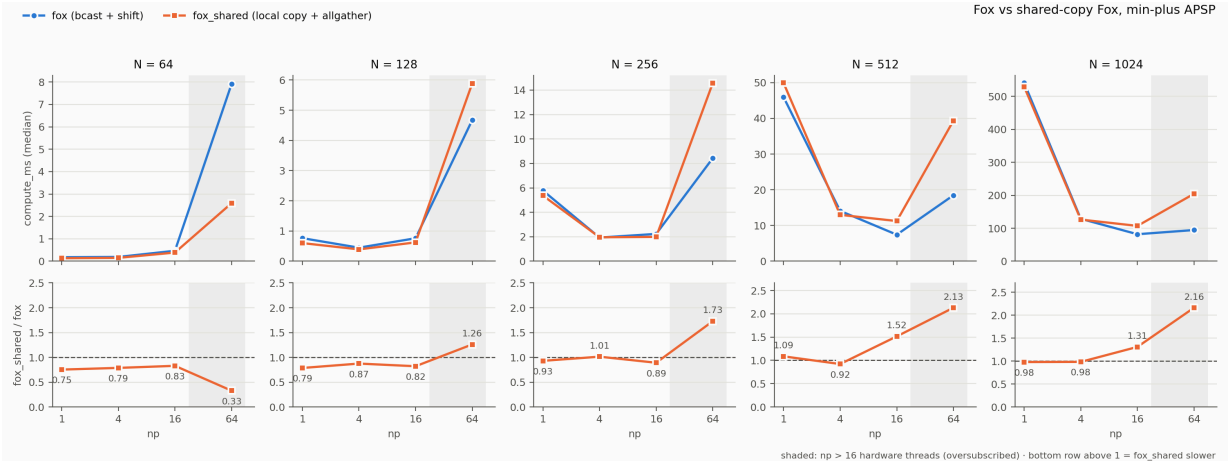


Figure 1. Top: median time per P, one panel per N, each with its own scale. Bottom: `fox_shared` time divided by `fox` time; above 1, `fox_shared` is slower. Shaded: P above the 16 hardware threads.

Table 3. Median execution time in milliseconds, `fox` against `fox_shared`.

	$P = 1$		$P = 4$		$P = 16$		$P = 64$	
N	fox	shared	fox	shared	fox	shared	fox	shared
64	0.18	0.13	0.19	0.15	0.47	0.39	7.92	2.59
128	0.76	0.60	0.45	0.39	0.76	0.62	4.67	5.88
256	5.78	5.37	1.93	1.96	2.23	1.99	8.44	14.6
512	46.0	50.0	14.1	13.0	7.42	11.3	18.5	39.4
1024	541.3	529.4	128.6	126.3	81.9	107.1	94.6	204.6

For $N \leq 256$, `fox_shared` is slightly faster (0.8–0.9 of the time of `fox` at $P \leq 16$). The blocks are tiny there, so the cost of `fox` is the latency of its $2Q$ messages per product rather than their size, and one all-gather per squaring is cheaper. As N grows the volume takes over: at $P = 16$, `fox_shared` takes 1.52 times as long as `fox` for $N = 512$ and 1.31 times for $N = 1024$, which is the $Q/2 = 2$ times larger transfer, not hidden behind computation. At $P = 4$ the two are within a few percent, since $Q/2 = 1$: both move the same amount of data. At $P = 1$ both run the same single local product.

5.4 Discussion

[TODO: Discuss Table 1 and Table 2: overhead of $P = 1$ against `seq`; scaling with P for each N; where communication starts to dominate (small N, large P); effect of using several nodes; effect of early termination.]

6 Conclusions

[TODO: Summary of results and possible improvements.]